

Data modyfikacji: sobota, 18 kwietnia

Termin oddania: środa, 29 kwietnia 23:59

Jako że listę zamieszczam dopiero w sobotę, 18 kwietnia, to termin jej wykonania jest na kolejny wtorek, czyli do 28 kwietnia. Tym samym, przesuwamy harmonogram zajęć o kolejny tydzień (poprzednio były przesunięte o tydzień ze względu na przerwę świąteczną). Na kolejnym spotkaniu 21 kwietnia będzie powtórzenie z JavaScripta, Promisy, wbudowane funkcje, klasy. Na tej liście obowiązuje więc materiał z obu spotkań.

Zmiany odnośnie prowadzenia ćwiczeń

[Sprawdziłem i oceniłem](#) wasze rozwiązania do list 1 i 2, ale było to trochę niedeterministyczne ze względu na brak struktury w rozwiązaniach. Tym samym, postanowiłem ustalić reguły odnośnie oddawania rozwiązań do zadań, są następujące:

- 1) Termin oddania rozwiązań do środy, godz. 23:59, czyli dzień po następnym spotkaniu,
 - a) Jeśli więc są problemy odnośnie listy, to można zgłosić to na spotkaniu i razem przyjrzymy się jak rozwiązać dane zadanie,
 - b) Rozwiązania oddane po terminie, maksymalnie do terminu oddania następnej listy, będą miały liczbę punktów pomniejszoną o 4,
- 2) Rozwiązania wrzucamy na *jedno* repozytorium (do całego kursu), struktura takiego repo powinna wyglądać następująco:

```
lista1
  zad1
    index.html
    index.js
    package.json
    # (inne pliki projektu)
  zad2
    ...
  zad3.txt      # jeśli zadanie polega na wyjaśnieniu czegoś itp.
lista2
  # zawartość kolejnej listy
```

Tym samym, pliki/foldery w środku lista{#numer} oznaczają te zadania, które deklarujecie.

Przez to, że na spotkaniach nie omawiamy zbyt wiele list przez brak czasu, możecie robić notatki do list/zadań, tłumacząc czego się dowiedzieliście, tłumacząc rozwiązania, ich poprawność itp. To da mi trochę więcej kontekstu przy ocenianiu ich.

Jeśli zadanie jest oznaczone literą, rozwiązaniem może być jeden plik odpowiedniego formatu (jak .js w Zadaniu A. z tej listy), ale podzielonym komentarzami na odpowiednie podzadania.

- a) Wystarczy, że raz wyślecie mi linka do waszego repozytorium, jeśli nie jest publiczne to dając mi dostęp do odczytu ([mój github](#)), następnie wystarczy, że tylko napiszecie w DMach “Lista zrobiona”.

Treści zadań rozpoczynają się na kolejnej stronie

Zadanie A. (8 pkt)

Funkcje Fibonacciego

1. (1 pkt)

Zaimplementuj funkcję `fib(x)` zwracającą x -ty element ciągu Fibonacciego na dwa sposoby: iteracyjny i rekurencyjny.

2. (1 pkt)

Zaimplementuj funkcję `arrOfFib(x)`, zwracającą tablicę kolejnych wyrazów ciągu Fibonacciego, od pierwszego do x -tego włącznie. Rozwiązanie powinno wykorzystywać tablicę do wyliczania kolejnych wartości

3. (1 pkt)

Zaimplementuj funkcję `selectFib(xs)`, zwracającą pofiltrowaną tablicę `xs`, zawierającą wyłącznie wyrazy ciągu Fibonacciego.

Wskazówka

Wystarczy że wyliczysz największy element tablicy `xs`, następnie policzysz wyrazy ciągu Fibonacciego aż do osiągnięcia tego maksimum, później pofiltrujesz tablicę `xs` względem funkcji charakterystycznej tablicy.

4. (1 pkt)

Zaimplementuj funkcję `shiftFib(x)`, zwracającą funkcję typu $(y) \Rightarrow ys$, która zaaplikowana na y zwraca tablicę wyrazów Fibonacciego do wyrazu y -tego, gdzie każdy element jest przesunięty o x .

Wskazówka 1

Funkcja powinna mieć taką semantykę:

$$(\text{shiftFib}(x))(y) = \text{arrOfFib}(y) \cdot \text{map}(z \Rightarrow z + x)$$

Wskazówka 2

Funkcja może zwracać callback (funkcję anonimową, lambda function) w następujący sposób:

```
function addXtoNumber(x) {  
  return (y) => x + y  
}  
Wtedy addXtoNumber(2) zwróci funkcję (y) => 2 + y, czyli (addXtoNumber(2))(2) = 4
```

5.

(1 pkt)

Zaimplementuj funkcję `addendsOf(x)`, zwracającą parę (tablicę dwuelementową) operandów dodawania dla wyrazu ciągu. Np. dla $F_4 = 5 = F_2 + F_3 = 2 + 3$, wynikiem powinna być `[2, 3]`. Dla liczby nie będącej elementem ciągu Fibonacciego zwróć `[]`.

6.

(2 pkt)

Zaimplementuj funkcję `customFib(ini, (x, xs) => number)`, która tworzy nowy ciąg zdefiniowany rekurencyjnie i zwraca funkcję typu `(x) => number` o semantyce przypisania `x` do x -tego wyrazu zdefiniowanego ciągu.

Znaczenie argumentów funkcji:

- `ini` to tablica początkowa, potrzebna do wyliczenia kolejnych wyrazów. Czyli dla klasycznego ciągu Fibonacciego będzie to `[1, 1]`, ponieważ kolejne wyrazy rekurencyjnie zależą od dwóch poprzednich. Długość tej tablicy musi więc wynosić ilość wyrazów zależnych we wzorze ogólnym wyrazu ciągu.
- `(x, xs) => number` to argument oznaczający funkcję, który przyjmuje `x`, czyli obliczany indeks wyrazu, oraz `xs` czyli obecny stan tablicy cache, a zwraca obliczony x -ty wyraz ciągu.

Klasyczna implementacja ciągu Fibonacciego wyglądałaby tak:

```
const fib = customFib([1,1], (x, xs) => xs[x-2] + xs[x-1])
```

Dla wartości `x` mniejszych od zera, funkcja może zwracać cokolwiek, np. `null`.

7.

(1 pkt)

Zaimplementuj funkcję `addendsOfArr(xs)` zwracającą obiekt typu `number: [number, number]`, czyli taki obiekt gdzie kluczami są liczby, a wartościami - pary liczb, gdzie para liczb odpowiada operandom używanym w celu obliczenia wyrazu klucza.

Wskazówka

Wystarczy, że najpierw połączysz tablicę wejściową `xs`, aby zostawić tylko te elementy, które są poprawnymi wyrazami ciągu Fibonacciego, a następnie zmapować elementy w parę `(element, [operand])`, gdzie `[operand]` można otrzymać z już dodanej funkcji z punktu 5.. Następnie użyj funkcji `Object.fromEntries()`, aby przekształcić tablicę par (klucz, wartość) w prawidłowy obiekt.

Zadanie B. (8 pkt)

Oceny uczniów

Dane są oceny uczniów klasy 5A z czterech przedmiotów: jpolski, matematyka, historia, wf w obiekcie oceny:

```
const oceny = {
  jpolski: {
    Jan: [4,3,4.5,4,2],
    Marek: [1,2,3,3,1.5],
    Pola: [5,4,6,5,5,4.5],
    Bartek: [3,5,3,2,5]
  },
  matematyka: {
    Jan: [3, 4, 2, 3, 3.5],
    Marek: [2, 2, 1, 3, 2],
    Pola: [6, 5, 5, 6, 5],
    Bartek: [4, 3, 4, 4, 3]
  },
  historia: {
    Jan: [4, 5, 4, 4, 5],
    Marek: [3, 2, 3, 4, 2],
    Pola: [5, 5, 6, 5, 5],
    Bartek: [2, 3, 2, 3, 3]
  },
  wf: {
    Jan: [5, 4, 5, 5, 4],
    Marek: [4, 4, 3, 5, 4],
    Bartek: [6, 6, 5, 6, 6]
  }
}
```

Uwaga: Za rozwiązania z zhardcodowanymi nazwami przedmiotów, czy imionami uczniów można otrzymać maksymalnie połowę punktów.

1.

(1 pkt)

Zaimplementuj funkcję `srednie(imie)` która liczy średnie ocen z każdego przedmiotu dla danego ucznia. Dla ucznia którego nie ma w bazie - powinno zwrócić obiekt z wartościami wypełnionymi zerami.

A więc dla `srednie("Jan")`, wynikiem będzie

```
{
  jpolski: 3.5,
  matematyka: 3.1,
  historia: 4.4,
  wf: 4.6
}
```

a dla `srednie("Garfield")` wynikiem będzie

```
{
  jpolski: 0,
  matematyka: 0,
  historia: 0,
  wf: 0
}
```

W szczególności dla Poli, która nie ma oceny z wf, `srednie("Pola").wf = 0`, a nie `undefined`.

2.

(1 pkt)

Zaimplementuj funkcję `najlepsi(przedmiot)` która zwróci tablicę obiektów zawierających pola `imie` i `oceny`, więc `najlepsi(string)` jest typu `{name: string, oceny: number}[]`. Te obiekty są posortowane malejąco według średniej.

`najlepsi("wf")` powinno więc zwrócić

```
[
  {
    name: "Bartek",
    oceny: [6,6,5,6,6]
  },
  {
    name: "Jan",
    oceny: [5,4,5,5,4]
  },
  {
    name: "Marek",
    oceny: [4, 4, 3, 5, 4]
  }
]
```

Psycholodzy szkolni uważają, że rywalizacja w nauce jest zdrowa. Zlecili nam stworzenie aplikacji mobilnej, gdzie każdy uczeń może odnaleźć swoje miejsce w szeregu. W tym celu, szkoła zapewniła nam dodatkowe dane:

```
let uczniowie = [
  {
    id: 0,
    imie: 'Jan',
    nazwisko: 'Kowalski',
    pesel: '082210499323',
    adres: 'ul. Turkusowa 34/9, 34-999 Oława',
    telefon: '+48 125 362 234',
    email: 'mama_janka@o2.drop',
    bio: 'Nie lubię nauki. Nie wiem co tu robię. Chcę grać w grę. Jaką? Tomb Rider!'
  },
  {
    id: 1,
    imie: 'Marek',
    nazwisko: 'Nowak',
    pesel: '082210555323',
    adres: 'ul. Polna 2',
    telefon: '+48 500 600 700',
    email: 'marek@nowak.pl',
    bio: 'Lubię sport, ale szkoła jest trudna.'
  },
  {
    id: 2,
    imie: 'Pola',
    nazwisko: 'Zdolna',
    pesel: '082210444321',
    adres: 'ul. Jasna 5',
    telefon: '+48 111 222 333',
    email: 'pola@interia.pl',
    bio: 'Zawsze celująca!'
  },
  {
    id: 3,
    imie: 'Bartek',
    nazwisko: 'Szybki',
    pesel: '082210666777',
    adres: 'ul. Sportowa 1',
    telefon: '+48 999 888 777',
    email: 'bartek@sport.pl',
    bio: 'Biegam szybciej niż myślę.'
  }
];
```

3.

(1 pkt)

Zaimplementuj funkcję `getPublicProfile(user)`, która przyjmuje obiekt ucznia i zwraca jego kopię pozbawioną danych wrażliwych (`pesel`, `adres`, `telefon`). Pozostałe pola powinny zostać zachowane. Skonstruuj funkcję tak, aby nie modyfikowała oryginalnego obiektu.

4.

(2 pkt)

Zaimplementuj funkcję `getSubjectRanking(nazwaPrzedmiotu)`.

- Funkcja powinna zwrócić tablicę obiektów reprezentujących wszystkich uczniów z tablicy `uczniowie`, posortowaną malejąco według ich średniej z danego przedmiotu.
- Każdy element tablicy musi mieć strukturę: `{ type: "id", value: id_ucznia }`.
- Jeśli uczeń nie posiada ocen z danego przedmiotu (brak klucza w obiekcie `oceny`), jego średnia w tym rankingu powinna wynosić 0.

5.

(1 pkt)

Zaimplementuj funkcję `expandIds(rankingArray)`, która przyjmuje tablicę identyfikatorów (format z punktu 4) i zamienia obiekty typu `id` na pełne obiekty danych uczniów pobrane z tablicy `uczniowie`.

Uwaga: Wynikowa tablica musi zawierać wyłącznie dane publiczne (użyj funkcji z punktu 3).

6.

(2 pkt)

Zaimplementuj funkcję `getRanking(imieUcznia)`, która dla podanego imienia zwraca obiekt informujący o pozycji ucznia we wszystkich dostępnych przedmiotach.

- Kluczami obiektu są nazwy przedmiotów, a wartościami 0-indeksowalne miejsca (indeks w posortowanej tablicy rankingowej).
- Funkcja powinna dynamicznie iterować po przedmiotach w obiekcie `oceny`.

Przykładowy wynik dla `getRanking("Jan")`:

```
{
  jpolski: 1,      // Jan jest drugi na liście (indeks 1)
  matematyka: 3,  // Jan jest czwarty na liście (indeks 3)
  // ... pozostałe przedmioty
}
```